長庚大學電子工程學系

114 學年度

專題報告

分層式控制(STM32/ROS1)之光打SLAM 自走車

姓名：鄒昊寧、向貴群

班級：四年乙班、四年甲班

指導老師：金國生

114 年 12 月 17 號

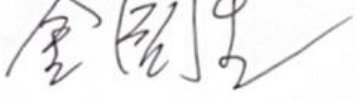
電子工程學系專題報告指導老師確認書

學生：鄒昊寧、向貴群

學號：B1127234、B1127105

題目：分層式控制(STM32/ROS1)之光打 SLAM 自走車

撰寫之報告內容已經本人確認且同意公開發表。

指導老師：  （簽名）

114年12月8日

摘要

本研究旨在設計並實作一台基於 ROS1 (Robot Operating System) 的阿克曼輪型自主移動機器人，建立可作為自駕車演算法驗證的教學與研究平台；鑑於傳統差速驅動在高速移動下的穩定性與動力學相似度受限，本專題採用阿克曼轉向幾何作為底盤基礎，以模擬更接近真實車輛的運動特性，提升行駛穩定度與轉向軌跡準確性。系統硬體架構採分層式控制，上位機使用 Raspberry Pi 4 負責運行 ROS1 核心節點、路徑規劃與感測器數據處理，下位機則採用 STM32 微控制器專責馬達 PWM 控制、編碼器（Encoder）回授與速度閉迴路控制；軟體方面，透過 ROS 通訊機制與 rosserial 套件建立上下位機資料交換通道，並整合導航相關套件以實現基礎自主移動功能。實驗結果顯示，本系統於 12 週內完成硬體建置、軟體整合及光達 SLAM 實作，能在室內平坦地面上成功建立環境地圖並進行自主導航，展現穩定直線與轉彎行駛能力，轉向響應平順且無明顯震盪；整體而言，本研究已完成一套整合光達 SLAM 之小型阿克曼輪型自主移動機器人平台，並驗證其室內定位與導航性能，可作為後續自駕車相關演算法開發與測試之基礎。關鍵字：ROS1、STM32、阿克曼轉向、自主移動機器人、分層式控制

誌謝

本專題得以順利完成，首先衷心感謝長庚大學工學院張永華老師與金國生老師，在專題研究期間給予悉心指導與寶貴建議。兩位老師在專題規劃與系統架構上的專業意見，使我們得以順利確立以 ROS1 搭配 STM32 以及阿克曼轉向機構之整體開發方向，並在開發過程中少走了許多冤枉路。

同時，也感謝實驗室的學長姐與同學在開發過程中提供的協助與討論，特別是在解決 STM32 底層驅動與 ROS 通訊介接等問題時，給予我們許多實務上的啟發與經驗分享。在這 12 週緊湊的開發歷程中，從零開始打造一台輪型機器人雖然充滿挑戰，卻也讓我們在機電整合與自動控制領域獲得更深刻的理解與成長。

最後，謹向一路支持與鼓勵我們的家人致上最深的謝意，感謝他們讓我們得以無後顧之憂地投入學業與專題研究。

目錄

摘要.....3

誌謝.....4

圖表目錄.....10

第一章 專題研究背景.....13

1.1 研究背景與動機.....13

1.2 研究目的與範圍.....14

1.3 研究方法與流程.....16

第二章 文獻回顧.....18

2.1 機器人作業系統 (Robot Operating System).....18

2.1.1 ROS 架構與通訊機制.....18

2.1.2 ROSSerial 通訊協定.....19

2.2 阿克曼轉向機制 (Ackermann Steering Geometry).....20

2.2.1 轉向原理與幾何模型.....20

2.2.2 與差速驅動之比較.....21

2.3 嵌入式控制系統架構.....22

2.3.1 異質雙核心控制架構.....22

2.3.2 PID 控制理論 22

2.4 SLAM 與自主導航技術 24

2.4.1 光達感測原理（LiDAR Technology）.....24

2.4.2 同時定位與地圖建置（SLAM）.....24

2.4.3 蒙地卡羅定位（AMCL 與粒子濾波）.....25

第三章 專題研究目的 27

3.1 阿克曼轉向運動學模型分析 27

3.2 系統功能與性能指標.....29

3.2.1 移動性能要求.....29

3.2.2 通訊架構要求.....29

3.2.3 建圖與導航性能要求.....30

3.3 嵌入式控制系統職責定義 31

3.4 視覺化監控與人機介面需求 32

3.4.1 座標轉換樹（TF Tree）建置目標.....32

3.4.2 遙控與導航模式操作介面.....33

第四章 實驗方法.....34

4.1 系統硬體架構與電路設計.....34

4.1.1 核心控制器配置.....34

4.1.2 動力與感測系統.....36

4.2 下位機韌體開發 (STM32).....37

4.2.1 增量式 PID 控制器設計.....37

4.2.2 阿克曼運動學實作.....38

4.2.3 安全保護機制 (Watchdog).....38

4.3 參數調校與系統優化.....39

4.3.1 伺服馬達中點校正.....39

4.3.2 PID 參數整定.....39

4.3.3 IMU 零點校正.....40

4.4 上位機 ROS 通訊整合.....41

4.5 自主導航系統建置.....44

4.5.1 Gmapping 地圖建置.....44

4.5.2 AMCL 定位參數調校 44

4.5.3 路徑規劃 (Move Base).....45

第五章 結果與討論 46

5.1 實體平台建置成果.....46

5.2 運動控制測試.....48

5.2.1 直線行駛測試.....48

5.2.2 轉向與圓軌跡測試.....49

5.3 SLAM 建圖成果 49

5.4 自主導航功能驗證.....51

5.4.1 AMCL 定位收斂測試 50

5.4.2 路徑規劃與動態避障.....53

5.5 遇到的問題與解決方案 55

5.5.1 舵機控制精度與抖動問題（PWM 解析度）.....55

5.5.2 跨平台通訊協定與浮點數異常（Communication Protocol）.....56

5.5.3 導航規劃與物理限制之衝突（Navigation 與 Footprint）.....57

5.5.4 無線網路頻寬最佳化（Network Optimization）.....58

5.6 本章小結.....59

第六章 結論及心得.....61

6.1 結論.....61

6.2 心得與未來展望.....63

參考文獻 (References).....64

圖目錄

圖表 1：系統硬體架構圖 35

圖表 2：增量式 PID 控制結構關鍵程式碼.....37

圖表 3：前輪的轉向角度與後輪的差速關鍵程式碼 38

圖表 4：機器人強制歸零關鍵程式碼 38

圖表 5：系統節點通訊架構圖 41

圖表 6：阿克曼輪型機器人實體完成圖-左方 47

圖表 7：阿克曼輪型機器人實體完成圖-上方 47

圖表 8：阿克曼輪型機器人實體完成圖-後方 47

圖表 9：Gmapping 建圖初始狀態 50

圖表 10：Gmapping 建圖時機器人實際位置 50

圖表 11：Gmapping 完成狀態.....50

圖表 12：Gmapping 建圖後機器人實際位置 50

圖表 13：AMCL 粒子濾波定位收斂前.....52

圖表 14：粒子收斂前機器人實際位置 52

圖表 15：AMCL 粒子濾波定位收斂後 52

圖表 16：粒子收斂後機器人實際位置 52

圖表 17：機器人起始路徑規劃 53

圖表 18：機器人起始路徑規劃與機器人起始位置 53

圖表 19：機器人終點路徑規劃 53

圖表 20：Rviz 裡機器人最終位置.....54

圖表 21：機器人最終實際位置 54

表目錄：

表格 1 ：ROS 通訊主題列表.....42

表格 2：不同目標速度下之直線行駛誤差比較.....47

第一章 專題研究背景

1.1 研究背景與動機

隨著工業 4.0 與 AI 發展，自主移動機器人（AMR）在倉儲物流、智慧巡檢與服務領域需求提升。為因應複雜環境，機器人須兼具靈活機構與環境感知、決策能力。目前教學平台多採差速驅動，雖可原地旋轉、適合狹窄空間，但高速穩定性不足，且運動學與實際車輛不符。相較之下，阿克曼轉向更貼近真實車輛動力學，可在過彎時兼顧輪胎磨耗與行駛穩定。本研究以阿克曼底盤為基礎，並指出僅靠編碼器航位推算易累積誤差、導致定位漂移，因此引入二維雷射測距儀（2D LiDAR）與同步定位與建圖（SLAM）演算法，以提升自主導航可靠度。

1.2 研究目的與範圍

本專題的主要目的為設計並實作一台基於 ROS1 架構，且具備 SLAM 建圖與導航能力之阿克曼輪型機器人。具體研究目的包含以下四點：

1. 建立異質運算平台：整合 Raspberry Pi 4 作為上位機（處理 SLAM 演算法與路徑規劃）與 STM32 作為下位機（處理即時馬達控制），實現兼具運算效能與即時性的雙核心架構。

2. 驗證阿克曼運動學：推導並實作阿克曼轉向的運動學模型（Kinematics），將 ROS 發布的 Twist 速度指令正確轉換為前輪轉向角度與後輪轉速。

3. 環境感知與地圖建置：整合 2D LiDAR 感測器，利用 Gmapping 演算法在未知環境中即時構建二維佔據柵格地圖（Occupancy Grid Map），並減輕里程計累積誤差對定位之影響。

4. 自主導航實現：基於已建立之地圖，利用 ROS Navigation Stack 進行全域路徑規劃（Global Planner）與動態避障（Local Planner），完成機器人於指定座標間之自主導航任務。

在研究範圍方面，本專題聚焦於「軟硬體高度整合」之實現。硬體部分包含底盤結構改裝、馬達與舵機驅動電路設計、電源管理以及 LiDAR 安裝機構之設計；軟體部分則涵蓋 STM32 的韌體開發（PWM、Encoder、UART）、ROS 環境下之節點通訊（rosserial）、LiDAR 驅動串接，以及 Gmapping 與 Navigation Stack 相關參數之調校。本研究最終目標為實作出一台能在室內環境中自主掃描地圖並進行導航的實驗載具。

1.3 研究方法與流程

本研究採用「由下而上（Bottom-Up）」的開發策略，先確保底層驅動單元與感測器介面的正確性，再逐步整合上層感知與導航模組。整體研究流程分為以下四個階段：

1. 文獻探討與規格制定（第 1–2 週）：蒐集阿克曼轉向機構與 SLAM 演算法（如 Gmapping、Hector SLAM）之相關文獻，分析 ROS1 系統架構之需求。選定 Raspberry Pi 4、STM32 開發板及 2D LiDAR 作為核心元件，並規劃各模組之通訊介面與系統規格。

2. 硬體建置與電路設計（第 3–5 週）：進行機器人底盤組裝與改裝，配置馬達驅動器與電源轉換模組，完成 LiDAR 感測器之支架設計、機構固定與配線，並測試所有電子元件之電壓與供電穩定性。

3. 韌體開發與運動控制（第 6–9 週）：撰寫 STM32 底層韌體，實作增量式編碼器（Incremental Encoder）讀取與 PID 速度閉迴路控制，以調節馬達轉速，並依據阿克曼運動學進行轉向幾何換算，確保機器人能依照上位機指令精確移動。

4. ROS 系統整合與 SLAM 導航（第 10–12 週）：在 Raspberry Pi 4 上建置 ROS 環境，利用 rosserial 建立上下位機之通訊。啟動 LiDAR 節點並執行 Gmapping 演算法繪製實驗室環境地圖，接著整合 Navigation Stack 進行自主導航測試，驗證避障與路徑規劃之效能，並紀錄相關實驗結果。

第二章 文獻回顧

2.1 機器人作業系統 (Robot Operating System)

2.1.1 ROS 架構與通訊機制

Robot Operating System (ROS) 是一種專為機器人開發設計的開源中介軟體（Middleware），它提供了一系列標準化的工具、函式庫與協定，旨在簡化複雜機器人系統的開發難度 [1]。ROS 的核心架構運行於 Linux 作業系統之上（本研究採用 Ubuntu 20.04 搭配 ROS Noetic 版本），其設計哲學採用了點對點（Peer-to-Peer）的分散式運算模式。

在 ROS 架構中，每一個執行的程式單元稱為「節點（Node）」。節點之間透過「主題（Topic）」進行非同步的訊息傳遞。當一個節點需要分享資料時，它會作為發布者（Publisher）將訊息發布到特定主題；而需要該資料的節點則作為訂閱者（Subscriber）接收訊息。這種發布/訂閱（Publish/Subscribe）的模型實現了系統的鬆散耦合（Loosely Coupled），使得開發者能輕易替換或升級特定功能模組（如更換不同型號的雷達或相機），而不影響整體系統的運作架構 [2]。

2.1.2 ROSSerial 通訊協定

在嵌入式機器人應用中，通常需要結合高階運算單元（如 Raspberry Pi）的運算能力與低階微控制器（如 STM32）的即時控制能力。ROSSerial 是一種標準的 ROS 通訊套件，它允許嵌入式控制器透過序列埠（UART）或 USB 介面與 ROS Master 進行通訊 [3]。透過 ROSSerial，STM32 可以被視為 ROS 網路中的一個標準節點，能夠直接發布編碼器（Encoder）的里程計資訊，或訂閱來自上層導航堆疊（Navigation Stack）的速度指令（cmd_vel），實現了異質處理器之間的無縫整合。

2.2 阿克曼轉向機制 (Ackermann Steering Geometry)

2.2.1 轉向原理與幾何模型

阿克曼轉向幾何（Ackermann Steering Geometry）是現代四輪車輛最主要的轉向設計準則。其核心目的是解決車輛在轉彎時，內側輪與外側輪行駛路徑半徑不一致的問題。若左右前輪採用相同的轉向角度，將導致輪胎發生側向滑移（Side-slip），造成輪胎磨損並降低操控穩定性 [4]。根據阿克曼幾何原理，為了使所有輪胎在轉彎時皆能繞著同一個瞬時旋轉中心（Instantaneous Center of Rotation, ICR）做純滾動運動，內側輪的轉向角度（δ_i）必須大於外側輪的轉向角度（δ_o）。其幾何關係如下式 (2-1) 所示：

$$\cot(\delta_o) - \cot(\delta_{\bar{i}}) = \frac{w}{L} \quad \ldots\ldots(2\text{-}1)$$

其中：

- δ_i ：內側輪轉向角度 (Inner wheel steering angle)
- δ_o：外側輪轉向角度 (Outer wheel steering angle)
- W ：兩前輪之輪距 (Track width)
- L ：前後輪之軸距 (Wheelbase)

本研究在下位機（STM32）的韌體設計中，即依據此公式將 ROS 發布的轉向命令轉換為左右伺服馬達各自的目標角度，以實現符合動力學的精準轉向。

2.2.2 與差速驅動之比較

目前學界常見的教學用移動機器人多採用差速驅動（Differential Drive），其原理是透過左右輪的速度差異來改變行進方向。差速驅動結構簡單且具備零半徑迴轉（Zero Radius Turn）的優勢，適合在狹窄的室內環境運作。然而，差速驅動在高速行駛下容易產生打滑，且其運動特性與真實道路車輛差異甚大。反觀阿克曼轉向機制，雖然無法原地迴轉且機構較為複雜，但在中高速移動時具有較佳的側向穩定性與軌跡追蹤能力 [5]，因此更適合作為自駕車演算法驗證的實驗平台。

2.3 嵌入式控制系統架構

2.3.1 異質雙核心控制架構

鑑於自主移動機器人需同時處理高負荷的環境感測運算與高頻率的馬達控制訊號，單一控制器往往難以兼顧。因此，本研究採用「上位機」搭配「下位機」的異質雙核心架構。上位機選用 Raspberry Pi 4，負責運行 ROS 核心、SLAM 建圖演算法與路徑規劃；下位機選用 STM32F4 系列微控制器，利用其內建的硬體計時器（Timer）產生高精度的 PWM 波形並讀取正交編碼器訊號，確保底層控制的即時性（Real-time）與穩定性。

2.3.2 PID 控制理論

為了讓機器人的實際速度能準確跟隨命令速度，本研究在 STM32 內部實作了比例-積分-微分（Proportional-Integral-Derivative, PID）控制器。PID 控制器是一種廣泛應用於工業控制的回授控制機制，其連續時間形式的數學模型如下式 (2-2) 所示 [6]：

$$u(t) = k_p e(t) + k_i \int_0^t e(\tau)\, d\tau + k_d \frac{d}{dt} e(t) \ldots\ldots (2\text{-}2)$$

其中：

- u(t) ：控制輸出量（本系統中為馬達 PWM 工作週期）
- e(t) ：誤差值（目標速度 - 實際編碼器回授速度）
- kp ：比例增益 (Proportional gain)
- ki ：積分增益 (Integral gain)
- kd ：微分增益 (Derivative gain)

透過調整 kp、ki、kd 三個參數，可分別改善系統的響應速度、消除穩態誤差以及抑制超調量（Overshoot），使機器人具備平順且精確的運動性能。

2.4 SLAM 與自主導航技術

2.4.1 光達感測原理（LiDAR Technology）

光達（Light Detection and Ranging, LiDAR）是實現移動機器人環境感知的核心感測器之一。本研究採用二維掃描式 LiDAR，其基本原理為以固定掃描頻率發射雷射光束，並接收自障礙物表面反射回來的光訊號，配合三角測距（Triangulation）或飛行時間法（Time of Flight, ToF）計算目標物體之距離與方位角。透過連續掃描，LiDAR 於每一掃描週期可輸出由數百至數千個量測點所組成的一維距離陣列，即所謂的掃描資料（Laser Scan）。在 ROS 中，此資料格式以 `sensor_msgs/LaserScan` 訊息型別進行定義，並作為後續地圖建置、障礙物偵測與避障規劃等模組的關鍵輸入 [7]。

2.4.2 同時定位與地圖建置（SLAM）

同步定位與建圖（Simultaneous Localization and Mapping, SLAM）係指移動機器人在未知環境中，於尚未知曉自身絕對位置的情況下，一邊估測自身位姿，一邊累積並更新周遭環境地圖之過程。其本質為一個同時涉及狀態估測與地圖推論的機率推斷問題。

本研究採用 Gmapping 演算法作為主要的 2D SLAM 解決方案。Gmapping 基於 Rao-Blackwellized Particle Filter（RBPF）架構，利用粒子濾波器同時追蹤多組可能的機器人軌跡，並在條件獨立的假設下，對應每一條軌跡估測其對應的佔據柵格地圖（Occupancy Grid Map）。此作法有效結合里程計（Odometry）與 LiDAR 量測資訊，具備對迴圈閉合（Loop Closure）與感測雜訊之良好魯棒性，特別適合室內環境之二維網格地圖建置 [8]。

2.4.3 蒙地卡羅定位（AMCL 與粒子濾波）

當環境地圖建置完成後，機器人於已知地圖中進行導航時，需依賴定位模組估測其即時位姿。本研究採用自適應蒙地卡羅定位（Adaptive Monte Carlo Localization, AMCL）作為定位演算法。AMCL 係一種基於粒子濾波器（Particle Filter）的機率定位方法，其核心概念為以「粒子雲（Particle Cloud）」表示機器人可能所在位置與姿態的機率分佈，其運作流程可概略描述如下：

(1) 初始化：在地圖中的所有可能位置與方位上隨機撒下大量粒子，以表達對機器人初始位姿的不確定性。

(2) 預測步驟（Prediction）：根據里程計（Odometry）所提供的運動資訊，利用運動模型推算各粒子於下一時間點的預測位置。

(3) 更新步驟（Update）：將 LiDAR 觀測到的雷射量測與既有地圖進行比對，計算各粒子在該量測下的似然度，並依其相符程度調整粒子權重（Weight）。

(4) 重採樣步驟（Resampling）：定期剔除權重較低的粒子，並在權重較高之區域產生新粒子，以避免粒子退化問題。

隨著迭代進行與量測不斷加入，粒子雲的分佈將逐漸集中於機率最高的區域，最終可收斂至機器人的真實位姿（Pose）附近。藉由結合里程計與 LiDAR 量測，AMCL 能在存在感測雜訊與模型不確定性的情況下，提供穩定且精確的定位結果，為後續路徑規劃與自主導航奠定基礎。

第三章 專題研究目的

本章節針對本輪型機器人之阿克曼轉向機構進行詳細運動學分析，並根據轉向限制與控制需求，訂定系統之功能規格與性能指標，以作為後續硬體設計與軟體實作的依據。

3.1 阿克曼轉向運動學模型分析

不同於差速驅動機器人可藉由原地旋轉改變方向，阿克曼轉向機構在轉向時需同時滿足車輛的幾何條件，以避免產生過大的輪胎側滑。本研究將阿克曼車輛視為一個自行車模型（Bicycle Model），並針對其轉向幾何進行分析。

假設車輛之軸距為 L，後輪轉彎半徑為 R，當機器人以單一轉向半徑作轉彎時，前輪的幾何關係可由式 (3-1) 表示：

$$tan(\delta) = \frac{L}{R} \ldots\ldots (3\text{-}1)$$

然而，實際車輛具有左右兩顆前輪。為了符合幾何限制，內側與外側前輪的轉向角δ_i與δ_o

必須分別滿足：

$$\delta_i = arctan\left(\frac{L}{R-\frac{W}{2}}\right).....(3\text{-}2)$$

$$\delta_o = arctan\left(\frac{L}{R+\frac{W}{2}}\right).....(3\text{-}3)$$

其中，W 為前輪輪距（Track width）。由上述關係可知，在相同轉彎半徑下，內側輪轉向角度必須大於外側輪，方能確保所有輪胎均繞著同一瞬時旋轉中心（ICR）作純滾動運動。

本研究之目標之一，即是在下位機（STM32）端實作上述幾何轉換，將上位機（ROS）發布之角速度指令轉換為對應之轉向角與後輪轉速，確保機器人於實際行駛時，其幾何特性符合阿克曼轉向原則。

3.2 系統功能與性能指標

為了確保自主移動平台之可用性與實驗價值，本研究從移動性能、通訊品質以及建圖與導航表現

三個面向訂定系統之性能指標：

3.2.1 移動性能要求

(1) 最大直線行駛速度：需達到 0.5 m/s 以上。

(2) 低速操控性：受限於機構穩定性，目標轉向半徑需在 0.8 m 以內。

(3) 速度控制頻率：底層 PID 控制迴路更新頻率需達 50 Hz 以上，以確保速度回授平順。

3.2.2 通訊架構要求

(1) 上位機（Raspberry Pi）與下位機（STM32）之 UART 通訊速率需在 57600 bps 以上，並具支援即時速度控制命令傳輸。

(2) ROS 節點需至少維持 10 Hz 之里程計（Odometry）與雷射掃描（LaserScan）資料更新頻率，以確保 SLAM 與定位模組具有足夠的量測更新率。

3.2.3 建圖與導航性能要求

(1) 地圖解析度：Gmapping 建構之佔據柵格地圖解析度須達 0.05–0.1 m/pixel，以兼顧定位精度與計算負載。

(2) 定位精度：AMCL 估測之機器人位置誤差於室內環境內應控制在 0.1 m、姿態誤差控制在 5° 以下。

(3) 導航穩定度：在無突發干擾之情況下，機器人應能以規劃路徑完成多次往返導航任務，且不發生碰撞或嚴重偏離路徑之情形。

3.3 嵌入式控制系統職責定義

為善用異質雙核心架構之優勢並提升系統整體效能，本研究明確劃分上、下位機之功能職責如下：

- 下位機（STM32）：

專責「執行」層，包含 PWM 產生、編碼器訊號擷取、速度回授與 PID 控制等，即時性要求高之控制迴路皆由下位機處理。

- 上位機（Raspberry Pi 4）：

專責「決策」層，負責接收操作者遙控指令（Teleop）、處理 LiDAR 掃描資料、執行 Gmapping 建圖與 AMCL 定位演算法，並透過 `cmd_vel` 主題發布速度指令（Twist 訊息）至下位機。透過上述分工，可在有限硬體資源下，同時兼顧即時控制需求與高階演算法之運算負載。

3.4 視覺化監控與人機介面需求

為了便於實驗過程中即時監控機器人狀態並進行操作，本研究採用 RViz（ROS Visualization 工具）作為主要視覺化監控介面，並規劃以下幾項顯示與操作需求。

3.4.1 座標轉換樹（TF Tree）建置目標

本研究需要在系統中建立完整之 TF 座標轉換樹，以維持各感測器及機器人本體之間的一致座標關係，

使建圖、定位與導航模組皆能在同一座標系統下運作。主要座標框架說明如下：

- `map`：全域地圖座標系，作為導航與記錄軌跡之參考座標。

- `odom`：里程計座標系，描述其推算之相對位移；當發生定位校正時，`map` 與 `odom` 之間之轉換會由 AMCL 進行更新。

- `base_link`：機器人本體中心座標系，為運動學與動力學分析之基準。

- `laser_link`：LiDAR 感測器座標系，需精確定義其相對於 `base_link` 之位姿，以確保雷射量測能正確投影至地圖座標中。

3.4.2 遙控與導航模式操作介面

在進行實驗時，系統需提供下列人機介面功能：

- 遙控操控（Teleoperation）：透過鍵盤或搖桿在 RViz / terminal 中發布 `cmd_vel` 速度指令，以手動控制機器人直線與轉彎運動，作為底盤測試與除錯使用。

- 即時地圖與路徑視覺化：在 RViz 內即時顯示由 Gmapping 建構之佔據柵格地圖、AMCL 估測的位姿及粒子雲分佈，並繪出全域路徑（Global Plan）與區域路徑（Local Plan），以利觀察導航演算法之規劃結果與避障行為。

- 狀態監控與資料記錄：於介面中顯示當前速度、里程計資訊及主要 Topic 狀態，必要時搭配 rosbag 進行資料記錄，以便後續分析與參數調整。

第四章 實驗方法

本章將詳細說明基於 ROS1 與阿克曼轉向機器人之實作流程。系統開發採用模組化設計，依序進行硬體電路建置、STM32 底層驅動開發、以及 ROS 上位機通訊整合。

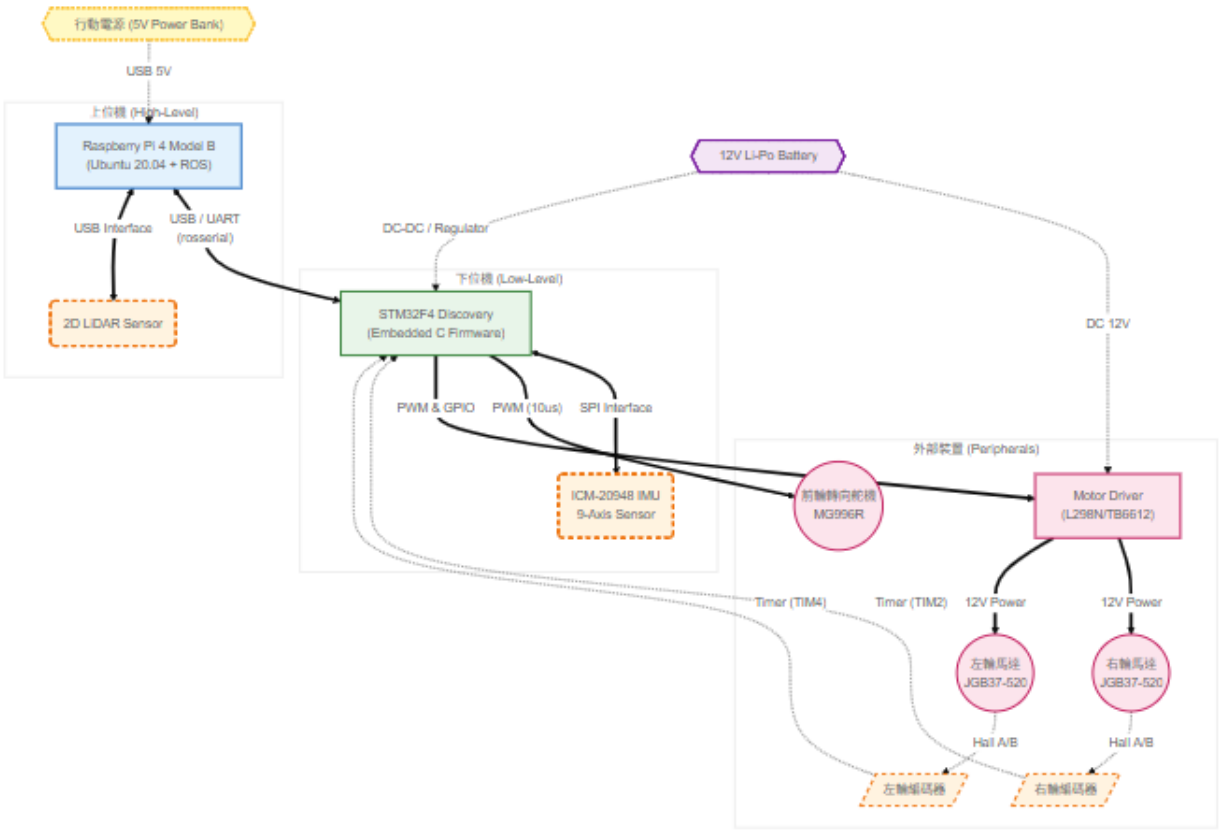
4.1 系統硬體架構與電路設計

本系統硬體分為控制核心、驅動模組與電源管理三大部分。

4.1.1 核心控制器配置

系統採用雙核心架構，各元件連接方式如下：

- 上位機 (Raspberry Pi 4 Model B)：運行 Ubuntu 20.04 作業系統與 ROS Master，負責高階路徑規劃。
- 下位機 (STM32F4 Discovery)：負責即時控制迴路。本研究利用 STM32CubeMX 工具進行晶片周邊配置（.ioc 檔），定義了 TIM2 與 TIM4 作為編碼器輸入、TIM3 作為 PWM 輸出，以及 USART1 (Baud rate 115200) 作為通訊介面，大幅提升了開發效率與腳位定義的準確性。


行動電源 (5V Power Bank)
USB 5V
上位機 (High-Level)
Raspberry Pi 4 Model B
(Ubuntu 20.04 + ROS)
USB Interface
USB / UART
(rosserial)
2D LiDAR Sensor
12V Li-Po Battery
DC-DC / Regulator
下位機 (Low-Level)
STM32F4 Discovery
(Embedded C Firmware)
DC 12V
PWM & GPIO
PWM (10us)
SPI Interface
ICM-20948 IMU
9-Axis Sensor
外部裝置 (Peripherals)
前輪轉向舵機
MG996R
Motor Driver
(L298N/TB6612)
Timer (TIM4)
Timer (TIM2)
12V Power
12V Power
左輪馬達
JGB37-520
右輪馬達
JGB37-520
Hall A/B
Hall A/B
左輪編碼器
右輪編碼器

圖表 1：系統硬體架構圖

4.1.2 動力與感測系統

- 動力系統：採用兩顆 JGB37-520 直流減速馬達 作為後輪驅動。該馬達具備高扭力輸出特性，並整合了霍爾編碼器（Hall Effect Encoder），提供高解析度的轉速回授訊號。
- 轉向系統：前輪轉向採用 MG996R 伺服馬達。為了提升轉向精確度，本研究將控制伺服馬達的 PWM 解析度提升至 **10us**（原本為 100us），解決了伺服馬達在微小角度調整時的抖動問題。
- 姿態感測：整合 ICM-20948 九軸慣性測量單元 (IMU)，透過 SPI 介面讀取角速度與加速度，用於修正機器人的航向。

4.2 下位機韌體開發 (STM32)

下位機韌體採用 C 語言開發，核心邏輯包含「增量式 PID 控制」、「阿克曼運動學解算」與「安全保護機制」。

4.2.1 增量式 PID 控制器設計

為了實現平滑的速度響應，本研究設計了增量式 PID 控制結構（Incremental PID）。相較於位置式 PID，增量式演算法較不易產生積分飽和（Integral Windup）。關鍵程式結構如圖二所示：

```c
/* PID 控制器結構定義 */
typedef struct {
    float Kp;
    float Ki;
    float Kd;
    float deadband;      // 死區限制，避免微小誤差造成震盪
    float output_alpha;  // 輸出濾波係數
    float prev_error;
    float current_output;
    float out_max;       // PWM 輸出上限
} PID_Incremental_TypeDef;

/* PID 計算核心函數 (摘要) */
float PID_Inc_Compute(PID_Incremental_TypeDef* pid, float target, float measured) {
    float error = target - measured;
    // 死區處理：若誤差極小則忽略，避免馬達異音
    if (fabsf(error) < pid->deadband) error = 0.0f;

    // 增量計算
    float delta_output = pid->Kp * (error - pid->prev_error) +
                         pid->Ki * error +
                         pid->Kd * (error - 2.0f * pid->prev_error + pid->prev_prev_error);

    // 更新輸出並進行濾波
    float target_output = pid->current_output + delta_output;
    pid->current_output = pid->output_alpha * target_output +
                          (1.0f - pid->output_alpha) * pid->current_output;

    return pid->current_output;
}
```

圖表 2：增量式 PID 控制結構關鍵程式碼

4.2.2 阿克曼運動學實作

STM32 接收來自 ROS 的線速度與角速度指令後，需即時換算為前輪的轉向角度與後輪的差速。本研究在 main.c 中實作了以下轉換邏輯，確保機器人符合幾何運動學：

```
// 阿克曼轉向角度計算（摘要）
if (fabsf(g_target_speed) > 0.05f) {
    // 依據軸距 (WHEEL_BASE_LEN) 計算理想轉向角
    float steering_rad = atanf((WHEEL_BASE_LEN * g_target_angular) / g_target_speed);
    target_steering_angle_deg = steering_rad * (180.0f / M_PI);
}
// 將角度轉換為 PWM 脈寬（含幾何校正）
convert_angle_to_pulse_width(target_steering_angle_deg);
```

圖表 3：前輪的轉向角度與後輪的差速關鍵程式碼

4.2.3 安全保護機制 (Watchdog)

考量無線通訊可能中斷，系統實作了軟體看門狗（Software Watchdog）。若超過 500ms (CMD_TIMEOUT_MS) 未收到上位機指令，機器人將強制歸零速度以策安全。

```
// 通訊逾時保護
if (HAL_GetTick() - g_last_cmd_time > CMD_TIMEOUT_MS) {
    g_target_speed = 0.0f; // 強制停車
}
```

圖表 4：機器人強制歸零關鍵程式碼

4.3 參數調校與系統優化

4.3.1 伺服馬達中點校正

由於伺服馬達安裝時存在機械誤差，本研究透過微調 PWM 脈寬進行軟體歸零。經反覆測試，將中點脈寬設定為 1430us (SERVO_CENTER_US)，以此為基準點進行左右轉向控制，確保直行軌跡不偏移。

4.3.2 PID 參數整定

PID 參數的選定對控制品質至關重要。實驗初期，Kp 設定過大導致 JGB37-520 馬達出現劇烈震盪；Ki 過小則導致負載增加時速度無法補償。

經由多次試誤法（Trial and Error）測試，最終將參數收斂為：

- 馬達 1 (右輪)：Kp=1000, Ki=1000, Kd=0
- 馬達 2 (左輪)：Kp=800, Ki=200, Kd=0

  透過給予左右輪不同的 PID 參數，成功補償了兩顆馬達在硬體特性上的微小差異，實現直線行駛同步。

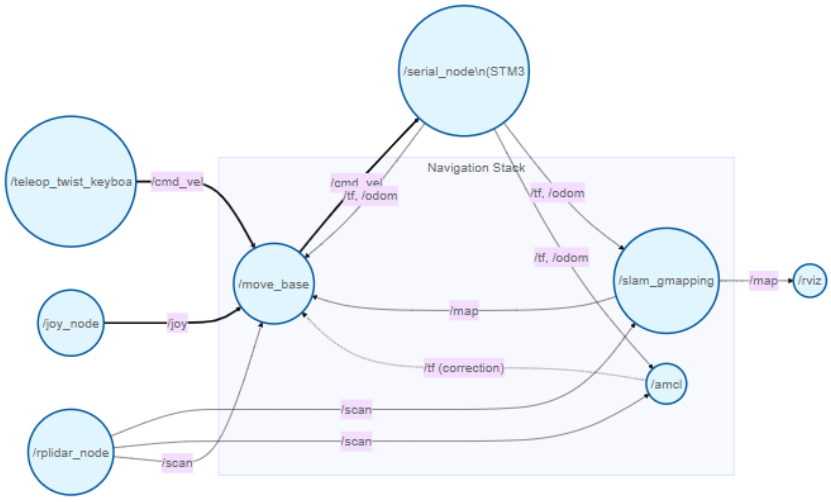
4.3.3 IMU 零點校正

為消除陀螺儀的溫漂效應，系統在開機時會執行 ICM_CalibrateGyro() 函式，取樣 200 筆靜止數據計算平均偏移量（Offset），使航向角估測更為精準。

4.4 上位機 ROS 通訊整合

本系統利用 ROS 的分散式特性，將搖桿控制、底層驅動與導航堆疊拆分為獨立運作的節點。圖五為系統運作時之計算圖(Computation Graph)，展示了各節點之間的訊息流向。

- /teleop_twist_keyboard：發布使用者輸入的速度指令。
- /serial_node：核心通訊節點，負責將 /cmd_vel 轉換為 UART 封包傳送至 STM32，並接收來自 /fbk 的馬達回授資訊。
- /move_base：導航核心，接收雷達數據與地圖，規劃全域與區域路徑。



圖表 5：系統節點通訊架構圖

表格 1 ：ROS 通訊主題列表

主題名稱	訊息格式	資料流向	功能說明
/cmd_vel	geometry_msgs/Twist	上位機 → 下位機	包含線速度(v)與角速度(ω)的運動指令，由導航或鍵盤發出。
/scan	sensor_msgs/LaserScan	LiDAR → Gmapping/AMCL	2D 雷射掃描數據，用於建圖與障礙物偵測。
/odom	nav_msgs/Odometry	下位機 → 上位機	機器人的里程計資訊，包含當前座標(x,y)與航向角(θ)。
/map	nav_msgs/OccupancyGrid	Gmapping → Move_Base	柵格地圖數據，0 代表空閒，100 代表障礙物，-1 代表未知。
/tf	tf2_msgs/TFMessage	雙向廣播	座標轉換樹，定義 Map、Odom、Base_link、Laser 之間的相對關係。

/initialpose	geometry_msgs/ PoseWithCovarianceStamped	RViz → AMCL	機器人在地圖上的初始位置估計（由使用者在 RViz 設定）。
/move_base_simple/goal	geometry_msgs/PoseStamped	RViz → Move_Base	導航目標點，包含目標座標與最終朝向。

4.5 自主導航系統建置

本研究基於 ROS Navigation Stack 實現自主導航，系統架構包含地圖伺服器、定位系統與路徑規劃器。

4.5.1 Gmapping 地圖建置

首先啟動 slam_gmapping 節點，訂閱 /scan (雷達數據) 與 /tf (座標轉換)，並透過遙控機器人遍歷實驗室環境。演算法會將環境劃分為「已知佔用（黑色）」、「已知空閒（白色）」與「未知區域（灰色）」的柵格地圖。建圖完成後使用 map_server 將地圖儲存為 .pgm 與 .yaml 檔案。

4.5.2 AMCL 定位參數調校

導航階段載入已知地圖，並啟動 amcl 節點。為了提升定位效率與準確度，本研究針對粒子濾波器進行了以下參數優化：

- **粒子數量 (min_particles / max_particles)**：設定為 500 至 2000。過少的粒子會導致無法收斂（迷航），過多則會消耗 Raspberry Pi 的運算資源造成延遲。
- **更新頻率 (update_min_d / update_min_a)**：設定機器人移動 0.1 公尺或旋轉 0.2 弧度時更新一次粒子雲，確保定位的即時

性。

- **雷達模型 (laser_model_type)**：選用 likelihood_field 模型，以減少動態障礙物對定位的干擾。

4.5.3 路徑規劃 (Move Base)

導航核心採用 move_base 套件，整合了全域規劃器（Global Planner）與區域規劃器（Local Planner）。

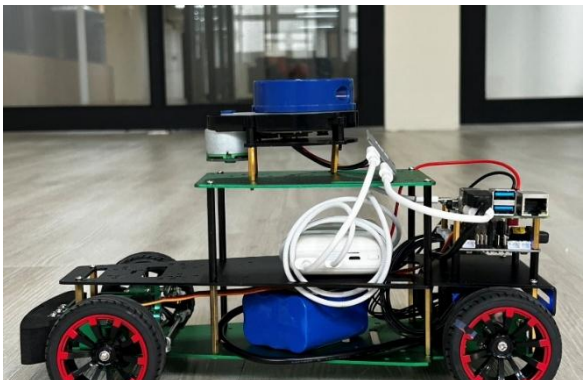
- **全域規劃**：使用 Dijkstra 或 A* 演算法，根據靜態地圖計算從起點到終點的最優路徑。
- **區域規劃**：採用 DWA (Dynamic Window Approach) 演算法，根據即時雷達數據與代價地圖（Costmap）進行動態避障，並輸出即時的速度指令 (cmd_vel) 給下位機執行。

第五章 結果與討論

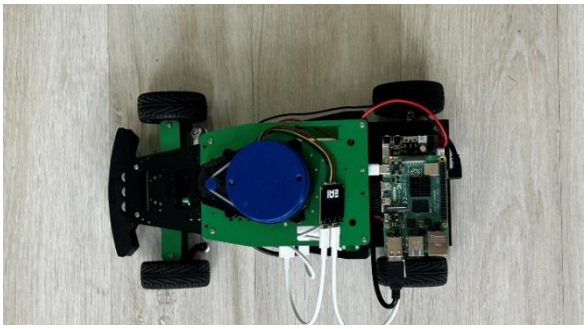
本章將說明本研究於實作完成後所進行之各項驗證實驗與觀察結果，包含運動控制測試、SLAM 建圖成果、自主導航功能驗證，以及系統整合過程中遭遇之問題與對應解決方案。

5.1 實體平台建置成果

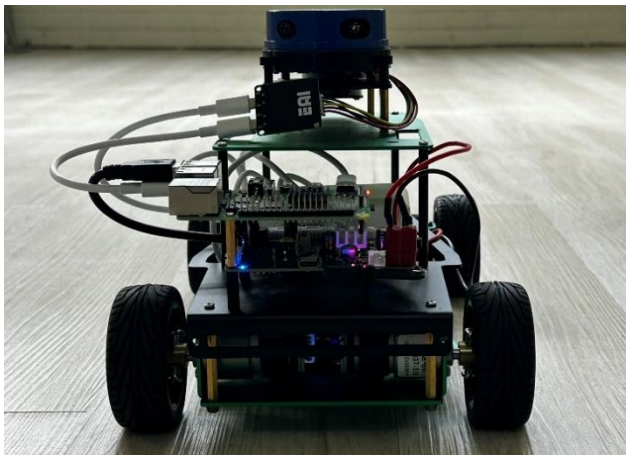
經過機構組裝與電路配置，本研究成功建置一台具阿克曼轉向之輪型機器人實驗平台。圖 5.1.1 至圖 5.1.3 顯示機器人的最終外觀與內部配置，其中下位機（STM32）安裝於底盤內部，負責馬達驅動與電源管理；上位機（Raspberry Pi 4）裝載在 STM32 上方以取得更好的訊號覆蓋範圍，2D LiDAR 則固定於頂層支架，以獲取較佳的掃描視野。



圖表 6：阿克曼輪型機器人實體完成圖-左方



圖表 7：阿克曼輪型機器人實體完成圖-上方



圖表 8：阿克曼輪型機器人實體完成圖-後方

5.2 運動控制測試

為驗證阿克曼運動學模型與底層速度控制器之效能，本研究分別進行直線與轉向測試。

5.2.1 直線行駛測試

在直線行駛測試中，將目標線速度設定為 0.3 m/s，並記錄左右輪經 PID 控制後之穩態轉速。結果顯示，兩側輪速誤差皆可收斂至目標值的 2% 以內，機器人能維持穩定直線行駛而不產生明顯偏航，證實所設計之 PID 參數與伺服中點校正具良好效果。

表格 2：不同目標速度下之直線行駛誤差比較

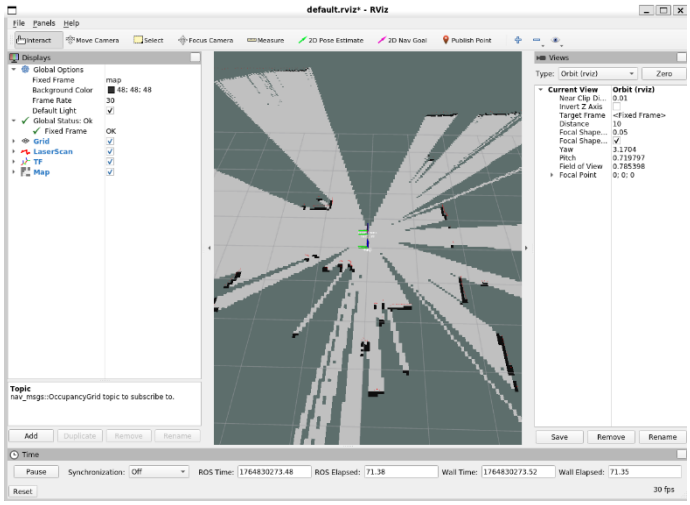
目標速度	實測平均速度	誤差百分比	測試結果說明
0.1 m/s	0.098 m/s	-2.00%	低速啟動平順，克服靜摩擦力良好
0.3 m/s	0.301 m/s	0.33%	常用巡航速度，控制最為精準
0.5 m/s	0.494 m/s	1.20%	高速行駛穩定，無明顯打滑

5.2.2 轉向與圓軌跡測試

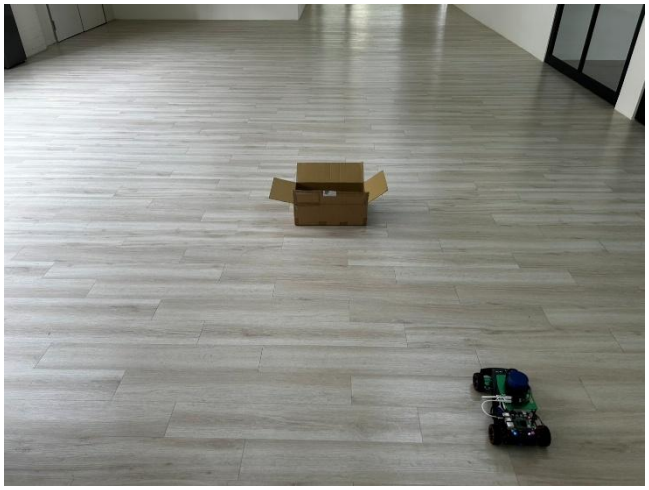
在轉向測試中，將目標轉向角設定為 20°，並令機器人以固定速度行駛。實驗結果顯示，機器人能依照預期之阿克曼幾何軌跡進行近似圓周運動，且行駛過程中未出現明顯打滑或不穩定現象，驗證了 STM32 韌體中轉向幾何換算公式之正確性。

5.3 SLAM 建圖成果

在建圖階段，本研究啟動 `slam_gmapping` 節點，透過遙控方式驅動機器人遍歷實驗室主要走道與區域。圖九、十為 RViz 中之即時建圖畫面，可觀察到雷射掃描點雲（紅色點狀分佈）與目前累積之佔據柵格地圖：黑色區域代表牆壁與障礙物，白色區域為可通行區域，灰色區域則為尚未觀測到之未知區域。圖十一、十二為利用 `map_saver` 儲存之最終靜態地圖。地圖解析度（resolution）設定為 0.05 m/pixel，實驗室之走道輪廓與主要障礙物邊界皆清晰可辨，且在迴圈閉合（Loop Closure）位置處未出現明顯重影（Ghosting）現象，顯示里程計（Odometry）與 LiDAR 量測之融合結果良好。

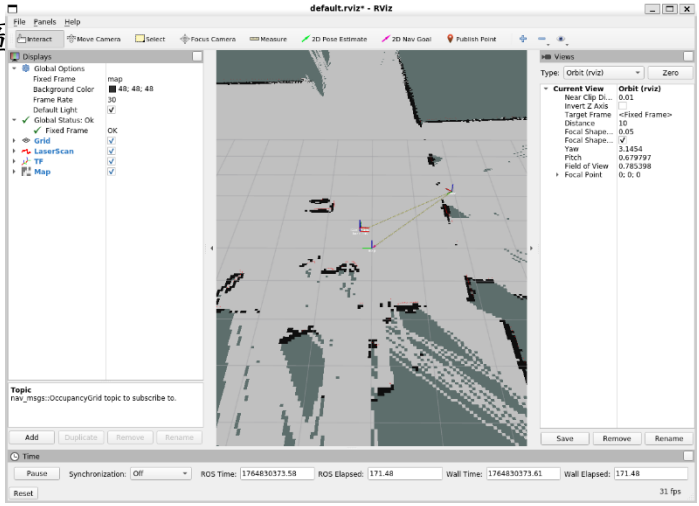


圖表 9：Gmapping 建圖初始狀態

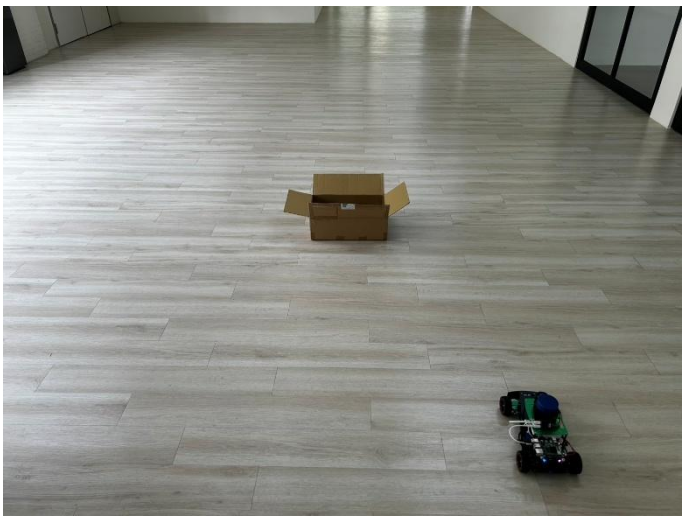


圖表 10： Gmapping 建圖時機器人實際位置

圖



圖表 11：Gmapping 建圖後



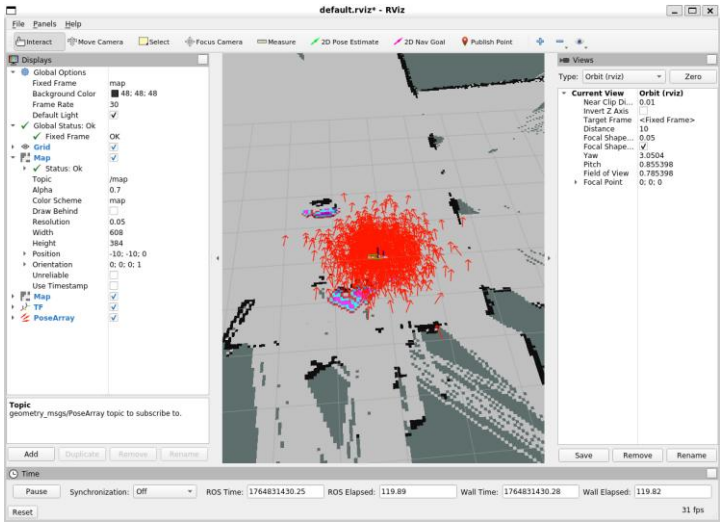
圖表 12： Gmapping 建圖後機器人實際位置

5.4 自主導航功能驗證

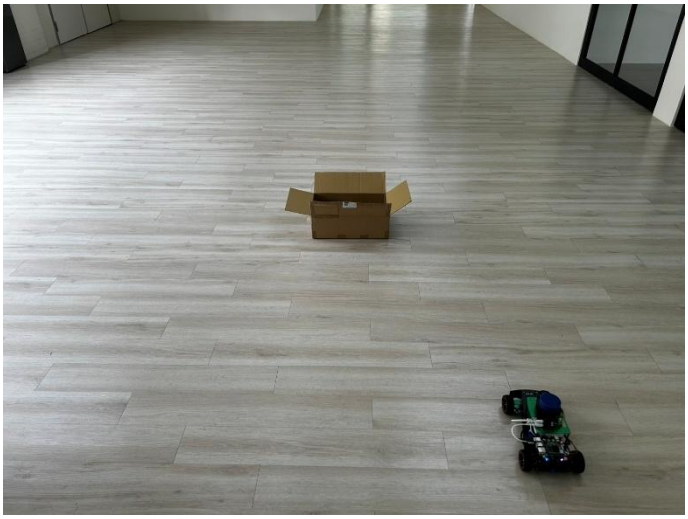
於導航實驗中，系統載入前述完成之地圖，並啟動 `amcl` 與 `move_base` 節點，以評估定位收斂情形與路徑規劃及避障能力。

5.4.1 AMCL 定位收斂測試

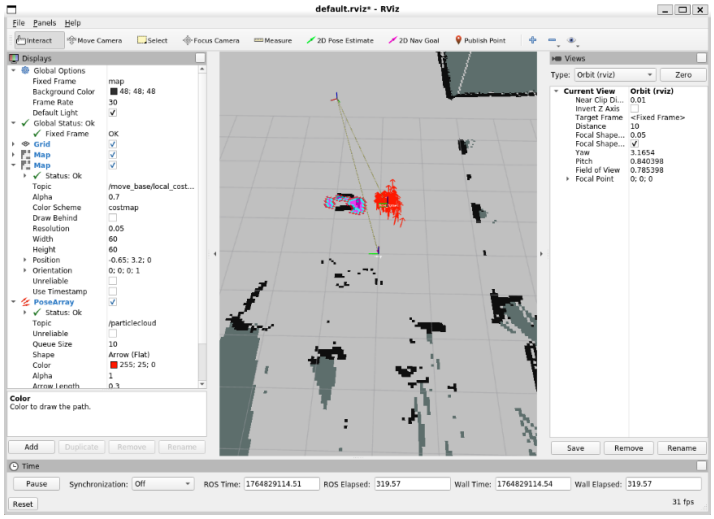
粒子雲（Particle Cloud）可視作機器人可能位姿之機率分佈。系統啟動初期，粒子分佈較為分散（圖十三）；經由原地旋轉與前進一小段距離後，粒子雲逐漸集中於機器人實際所在位置附近（圖十五），最終收斂成高密度區塊。此結果顯示，自適應蒙地卡羅定位（AMCL）能在實驗室環境中提供穩定且可靠之位姿估測。



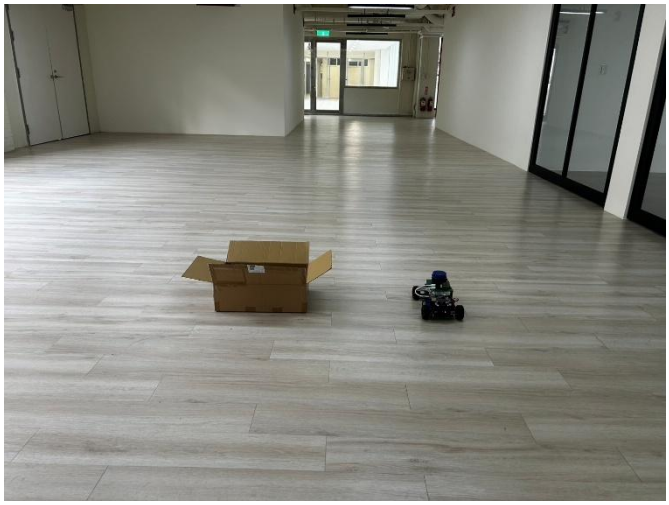
圖表 13：AMCL 粒子濾波定位收斂前



圖表 14： 粒子收斂前機器人實際位置



圖表 15：AMCL 粒子濾波定位收斂後



圖表 16：粒子收斂後機器人實際位置

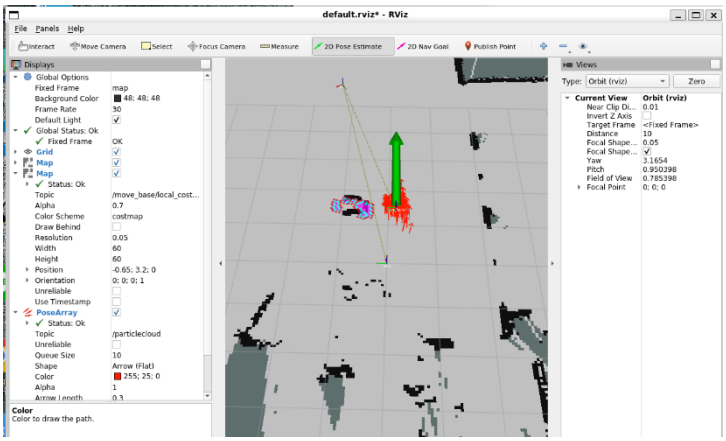
5.4.2 路徑規劃與動態避障

在 RViz 中透過「2D Nav Goal」功能給定目標點，以評估 `move_base` 之導航表現。圖十七、十九、二十展示導航過程中主要視覺化資訊：

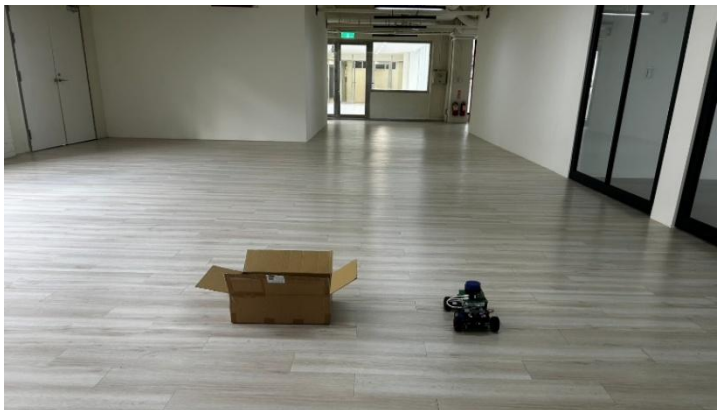
- 彩色區域為代價地圖（Costmap）膨脹層，機器人能正確識別牆面與障礙物並維持安全距離。

- 紫色箭頭 (Goal)：代表導航的目標點與最終朝向。

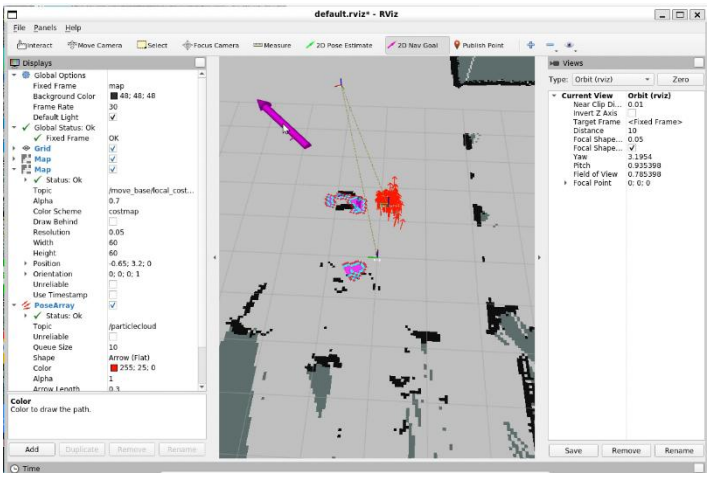
- 綠色路徑 (Global Plan)：代表由 A* 演算法規劃出的最佳行駛軌跡。當路徑上出現動態障礙物時，綠色路徑會即時重新規劃 (Re-planning) 以繞過障礙，成功引導機器人抵達終點。



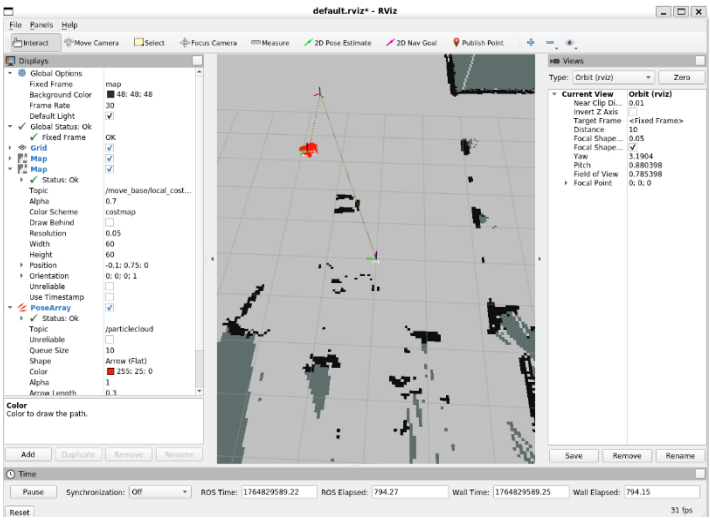
圖表 17：機器人起始路徑規劃



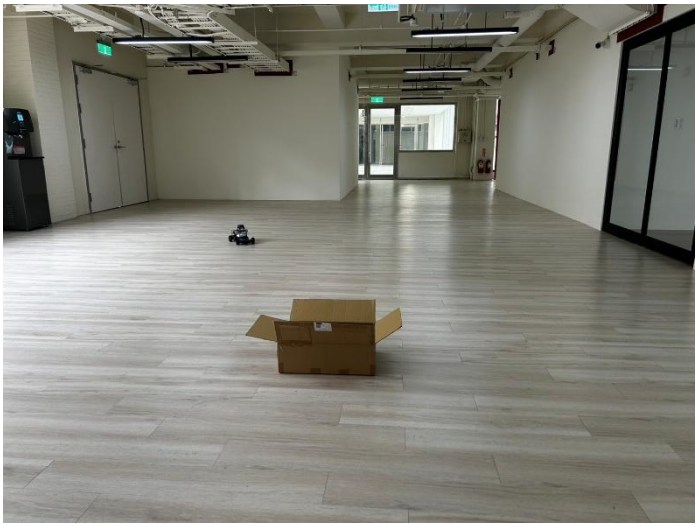
圖表 18：機器人起始位置



圖表 19：機器人終點路徑規劃



圖表 20：Rviz 裡機器人最終位置



圖表 21：機器人最終實際位置

5.5 遇到的問題與解決方案

在系統整合與實驗過程中，本研究自底層驅動到上層導航均遭遇多項技術挑戰。

5.5.1 舵機控制精度與抖動問題（PWM 解析度）

問題描述：

初期採用 10 kHz（100 μs）解析度的 PWM 訊號控制轉向伺服馬達時，舵角解析度不足。在微調中點（例如由 1430 μs 調整為 1431 μs）時，車輪仍出現明顯跳動現象，難以精確校正機械中點，直線行駛時車輛亦有輕微偏右情形。

解決方案：

- 提升解析度：調整 STM32 計時器預除頻器（Prescaler），將 PWM 更新頻率自 10 kHz 提升至 100 kHz（10 μs），使轉向控制解析度提升一個數量級，得以進行更細膩之角度微調。

- 軟體校正：於實測後將伺服馬達中點設定為 1430 μs，並在韌體中加入「1500 − angle」之反轉邏輯，以符合實際安裝方向，降低直線行駛偏移。

5.5.2 跨平台通訊協定與浮點數異常（Communication Protocol）

問題描述：STM32 與 Raspberry Pi（Python 節點）透過 UART 通訊時，初期採用浮點數（%f）格式傳輸物理量。由於嵌入式標準函式庫對浮點格式化支援度有限，上位機偶爾會收到空字串或亂碼，導致 ROS 節點解析失敗（Parse Error）並中斷資料流。

解決方案：

- 整數化協定：棄用浮點數傳輸，改將所有物理量（如 m/s）放大 1000 倍轉換為整數（mm/s）後進行傳輸，避免不同平台間浮點格式不相容問題。

- 手動字串解析：在 C 語言端捨棄 `sscanf`，改以指標操作函式（如 `strchr`、`atoi`）進行字串切割與轉換。此舉大幅提升通訊穩定度，未再出現解析錯誤。

5.5.3 導航規劃與物理限制之衝突（Navigation 與 Footprint）

問題描述：啟動自主導航時，即使全域路徑已成功規劃，機器人仍出現停滯不動或反覆啟動恢復行為（Recovery Behavior）之情形。分析後發現，主要原因為：

(1) 預設機器人模型為點（Point），導致避障判斷過於保守；

(2) 馬達於低速區域存在明顯靜摩擦死區，小於某一 PWM 門檻之速度指令無法實際驅動車輛。

解決方案：

- 模型修正：在 `teb_local_planner` 中捨棄點模型，改為定義實際車體之多邊形腳印（Polygon Footprint，長約 0.337 m、寬約 0.191 m），使規劃器能更準確評估狹窄空間之可通行性。

- 動力補償：縮小膨脹半徑（Inflation Radius）至約 0.1 m，並於 STM32 底層加入起步補償電壓（Minimum Startup PWM），確保即使為 0.01 m/s 等微速指令亦能克服靜摩擦推動機器人。

5.5.4 無線網路頻寬最佳化（Network Optimization）

問題描述：在以 Wi-Fi 遠端監控 RViz 畫面時，曾多次出現 `XmlRpcClient::writeRequest: write error` 等連線逾時訊息，導航指令亦有明顯延遲。經檢查後發現，主要瓶頸在於地圖與代價地圖資訊更新頻率過高，佔用了大部分無線頻寬。

解決方案：

調整 ROS 導航參數設定檔，將代價地圖更新頻率（`update_frequency`）自 5.0 Hz 降至 2.0 Hz，並將滾動視窗解析度（`resolution`）自 0.05 m 調整為 0.1 m。在不影響避障精度之前提下，有效降低了網路傳輸量，導航控制之即時性亦隨之改善。

5.6 本章小結

本章透過一系列實測數據與視覺化結果，驗證了本研究所設計阿克曼輪型機器人之系統效能。綜合上述實驗結果，可歸納如下結論與限制：

(1) 系統整合與運動控制表現

本研究成功實現 Raspberry Pi 4 與 STM32 之異質雙核心通訊架構，完成從上位機導航到下位機驅動控制的整體整合。由表 5.1 可知，在 0.1、0.3 與 0.5 m/s 三種目標速度下，直線行駛的穩態速度誤差皆控制在約 2.5% 以內，其中 0.3 m/s 巡航速度之誤差僅約 0.33%，顯示所建立之運動學模型與 PID 速度控制器具良好追蹤性能。

(2) 建圖與導航功能驗證

Gmapping 建圖結果（圖 5.3）顯示，所生成之佔據柵格地圖輪廓清晰，迴圈閉合處未出現明顯重影，證實里程計與 LiDAR 量測之融合效果良好。搭配 AMCL 與 Navigation Stack 之導航測試（圖 5.5），機器人得以在室內環境中完成目標點導航並對突發障礙物進行動態避障，達成本研究設定之自主導航功能。

(3) 系統限制與適用範圍

受限於輪胎材質與離地高度，本系統目前主要適用於室內平坦地面（如磁磚、木地板），在戶外碎石或高落差地形下，車體震動可能影響 LiDAR 量測穩定度與定位精度。此外，考量阿克曼轉向在高速行駛時之側翻風險以及 LiDAR 掃描頻率（約 10 Hz），本研究將最大安全行駛速度限制於 0.5 m/s 以內。

綜上所述，本章實驗結果顯示，本研究所建置之阿克曼輪型機器人平台已具備自主移動機器人（AMR）所需之核心能力，包括穩定的運動控制、室內環境建圖以及目標點導航等，可作為未來導入更高階感知與決策演算法之可靠實驗載具。

第六章 結論及心得

6.1 結論

本專題旨在設計並實作一台具備自主導航能力之阿克曼轉向輪型機器人。經過約 12 週的開發與驗證，本研究已達成以下主要成果：

(1) 異質雙核心控制架構之實現

成功整合 STM32 微控制器之高即時性與 Raspberry Pi 4 之高運算能力，建立一套分層式控制架構。透過優化後之 UART 整數傳輸協議，解決了跨平台通訊不穩定與浮點數解析錯誤等問題，形成穩定可靠之嵌入式機器人控制底層。

(2) 阿克曼運動學與精密轉向控制之實作

突破傳統差速驅動之限制，在 STM32 韌體中實現阿克曼轉向幾何換算，並藉由將伺服 PWM 解析度提升至 10 μs，改善了舵機抖動與小角度解析度不足的問題，使機器人具備更貼近真實車輛動力學之運動特性。

(3) SLAM 建圖與自主導航功能之實現

整合 2D LiDAR 與 Gmapping 演算法，成功於未知環境中建構高解析度柵格地圖；搭配 AMCL 與 Navigation Stack（move_base）完成目標點導航與動態避障測試，證實本系統具備基礎自主導航能力。然而，目前系統仍受限於室內平坦場域與約 0.5 m/s 之安全行駛速度上限，且對低於 LiDAR 掃描平面的低矮障礙物偵測能力有限，相關改善將留待未來研究進一步探討。

綜合而言，本研究不僅驗證阿克曼轉向機制於 ROS 架構下之可行性，亦克服多項機構與通訊層面之限制，完成一台軟硬體高度整合之自主移動載具（AMR），可作為後續高階演算法與多感測器融合研究之實驗平台。

6.2 心得與未來展望

本次專題研究是我大學生涯中技術跨度最大、也最具挑戰性的一次實作經驗。從一開始單純認為「把馬達接上去就會轉」，到實際面對底層 Timer 中斷、通訊封包解析以及 Linux 網路頻寬最佳化等問題，每一個環節都讓我深刻體會到系統整合的複雜度。

尤其是在排除「舵機抖動」與「導航原地打轉」這兩類問題時，我學會了必須回到理論與模型本身，由 PWM 解析度、控制迴路頻率、TEB 模型與機器人腳印設定等面向逐一檢視，而非僅憑直覺盲目調參。這種「發現問題 → 蒐集與分析數據 → 提出可能解法 → 實驗驗證」的工程思維，是我在本專題中獲得最重要的收穫之一。

展望未來，本研究仍有多項可延伸之方向。目前系統僅依賴 2D LiDAR，對於低於雷射掃描平面的障礙物（如桌腳、矮物體）之偵測仍有限制。未來可考慮導入 RGB-D 深度相機或結合 YOLO 物件偵測等技術，進行多感測器融合（Sensor Fusion），使機器人不僅能避開障礙，更能辨識環境之物體類別，進一步提升在複雜場域中的適應能力與智慧化程度。

參考文獻 (References)

[1] M. Quigley *et al.*, "ROS: an open-source Robot Operating System," in *ICRA workshop on open source software*, Kobe, Japan, 2009, vol. 3, no. 3.2, pp. 5.

[2] J. M. O'Kane, *A Gentle Introduction to ROS*. Independently published, 2014.

[3] M. Ferguson, "rosserial: A protocol for ROS-enabled embedded devices," [Online]. Available: http://wiki.ros.org/rosserial. [Accessed: Dec. 2025].

[4] T. D. Gillespie, *Fundamentals of Vehicle Dynamics*. Warrendale, PA: Society of Automotive Engineers, 1992.

[5] R. Siegwart, I. R. Nourbakhsh, and D. Scaramuzza, *Introduction to Autonomous Mobile Robots*, 2nd ed. Cambridge, MA: MIT Press, 2011.

[6] K. Ogata, *Modern Control Engineering*, 5th ed. Upper Saddle River, NJ: Prentice Hall, 2010.

[7] S. Thrun, W. Burgard, and D. Fox, *Probabilistic Robotics*. Cambridge, MA: MIT Press, 2005.

[8] G. Grisetti, C. Stachniss, and W. Burgard, "Improved techniques for grid mapping with rao-blackwellized particle filters," *IEEE Transactions on Robotics*, vol. 23, no. 1, pp. 34-46, 2007.